Semantic Segmentation with Dilated Networks

Final Project Presentation

Mohammadamin Alibakhshi

1) Problem, Motivation, and Goal

Semantic segmentation:

Assigns a class label to **every pixel**

- Different from a “cat” label for the whole image
- Different from box around objects

1) Problem, Motivation, and Goal

Dataset:

Cityscapes

- leftImg8bit_trainvaltest : 5000 images
- Fine and course annotations
- For annotations we used: gtFine

Fine annotations

Coarse annotations

1) Problem, Motivation, and Goal

Dataset:

Cityscapes

- leftImg8bit_trainvaltest : 5000 images
- Fine and course annotations
- For annotations we used: gtFine

Fine annotations

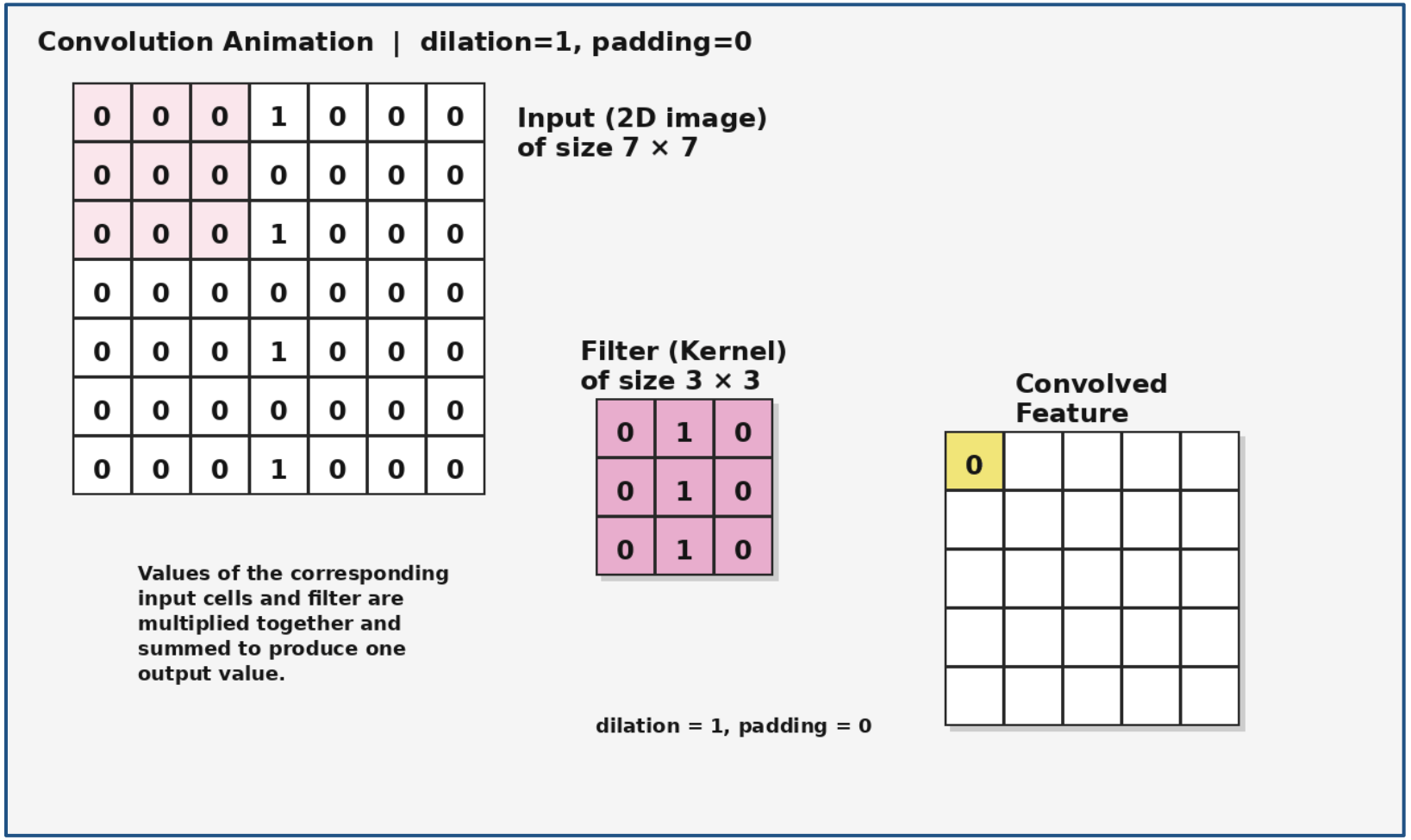
1) Problem, Motivation, and Goal

Convolution:

Weight sharing

- Shift-invariant
- Conv kernel

  = **response to an impulse**

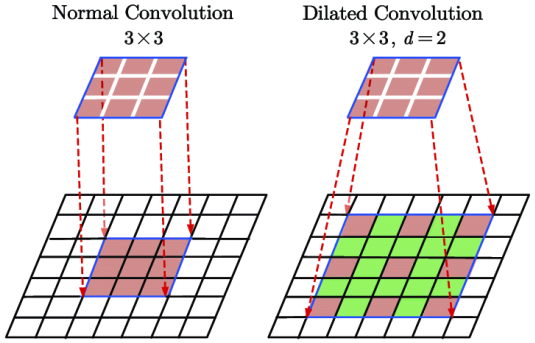


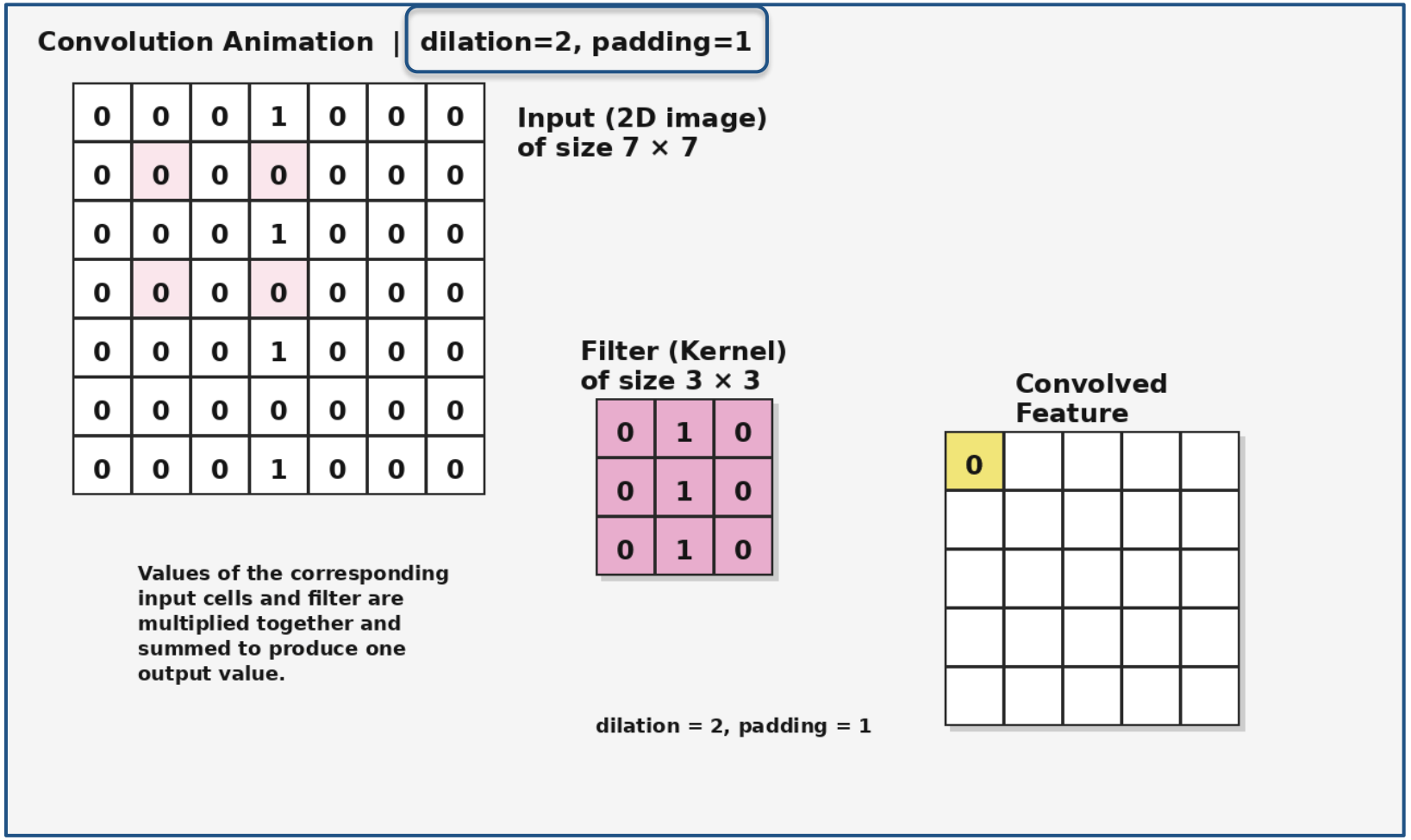
Dilated Convolution

- **Inserts gaps between kernel points**
- Increase receptive field
- Padding for output consistency
- Compare to Max pooling:

  lower spatial resolution
- While dilated conv:

  keeps higher spatial resolution



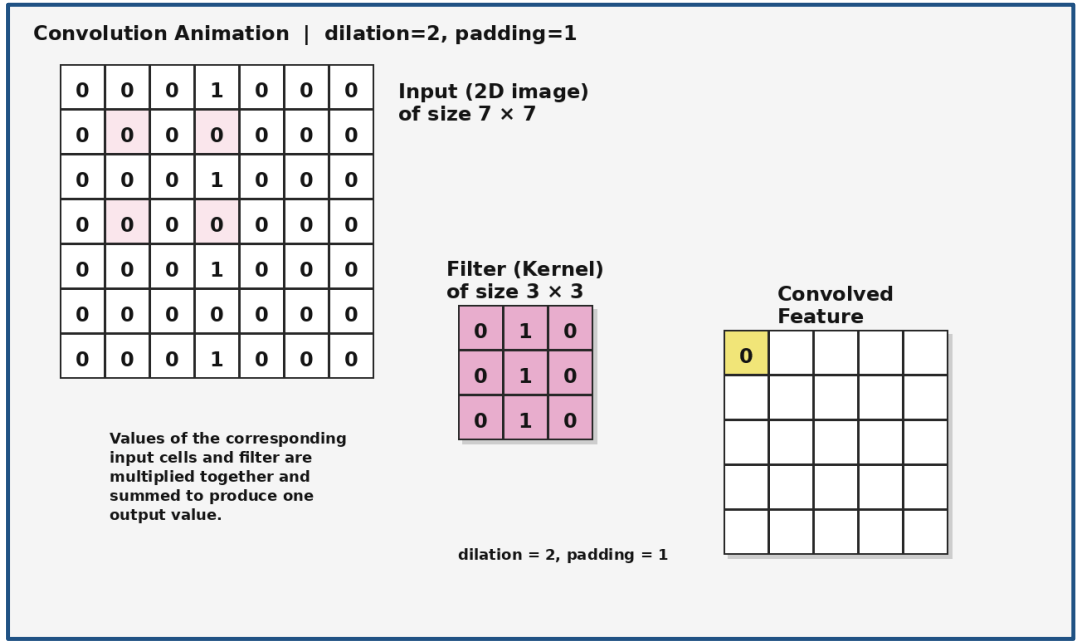


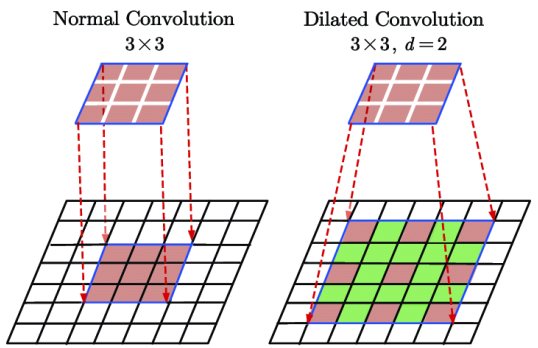
Dilated Convolution

- **Padding** for output consistency

$$\text{effective kernel} = (k-1)d + 1$$
$$\text{d=1: effective kernel} = (3-1)1 + 1 = 3$$
$$\text{d=2: effective kernel} = (3-1)2 + 1 = 5$$

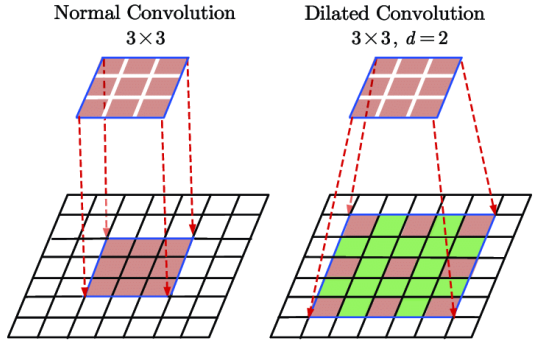
$$\text{padding} = \frac{(k-1)d}{2}$$
$$\text{d=1: padding} = \frac{(3-1)1}{2} = 1$$
$$\text{d=2: padding} = \frac{(3-1)2}{2} = 2$$
$$\text{d=3: padding} = \frac{(3-1)3}{2} = 3$$





1) Problem, Motivation, and Goal

- Task: Semantic segmentation for autonomous driving scenes.
- Question: **does dilation improve performance across architectures?**
- Method: compare baseline and dilated versions of four model families.
- Metric: Mean IoU over 100 test samples.



2) Dataset and Preprocessing

- Cityscapes fine labels (2,975 train + 500 validation images).
- Images size $3 \times 1024 \times 2048$.
- **19 training classes** with **ignore index** 255 for invalid labels.
- Pair leftImg8bit and gtFine labels

3) Models and Dilation Strategy

- Simple CNN and dilated Simple CNN (Manually implemented)
- VGG16 and VGG16 Dilated (Encoder imported from torch, decoder manually implemented)
- SegNet and SegNet Dilated (Structure inspired from SegNet but manually implemented)
- LRASPP and LRASPP Dilated (Imported from torch, but trained the weights)

- Same evaluation and training protocol used to isolate architecture effects.

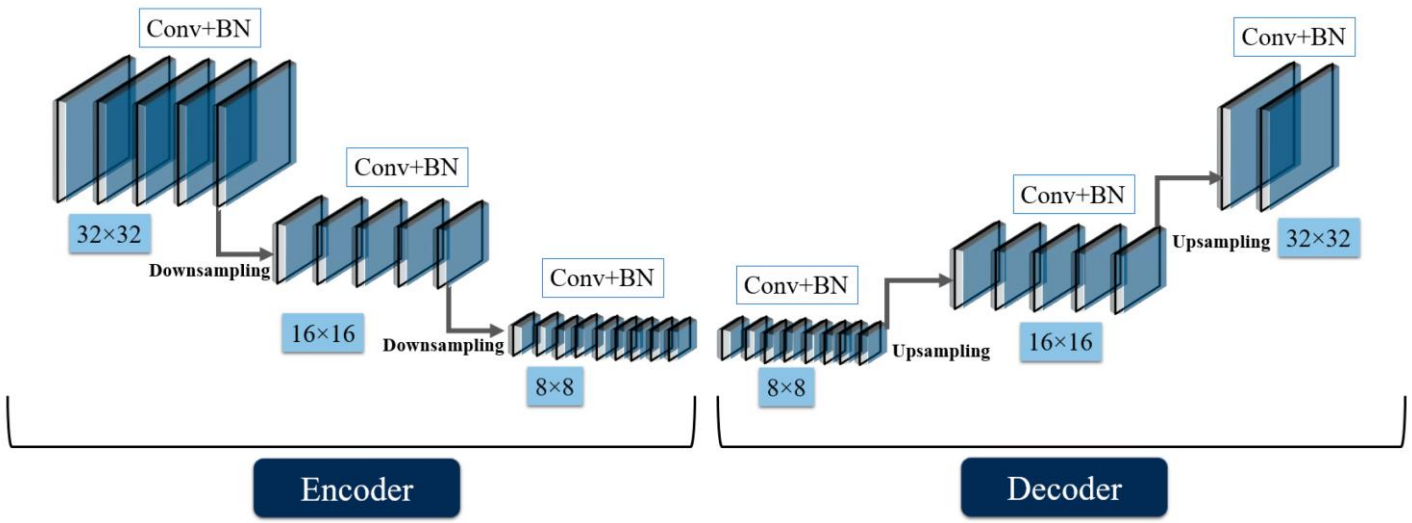
3) Models and Dilation Strategy

Simple CNN

- Why we need upsampling? output a mask prediction for every pixel

dilation=4, padding=4 to all 3 encoder convs

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 2, stride=2),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 2, stride=2),
            nn.ReLU(),
            nn.Conv2d(32, 19, 1),
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))
```



Architecture is not the same (Just for visualization)

Source: mdpi

3) Models and Dilation Strategy

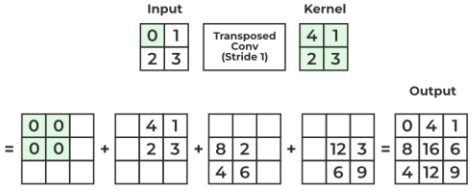
Simple CNN

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 2, stride=2),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 2, stride=2),
            nn.ReLU(),
            nn.Conv2d(32, 19, 1),
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))
```

Dilated Simple CNN

```python
class DilatedCNN(nn.Module):
    def __init__(self):
        super().__init__()
        dilation = 4
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=dilation, dilation=dilation),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=dilation, dilation=dilation),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=dilation, dilation=dilation),
            nn.BatchNorm2d(128),
            nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 2, stride=2),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 2, stride=2),
            nn.ReLU(),
            nn.Conv2d(32, 19, 1),
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))
```

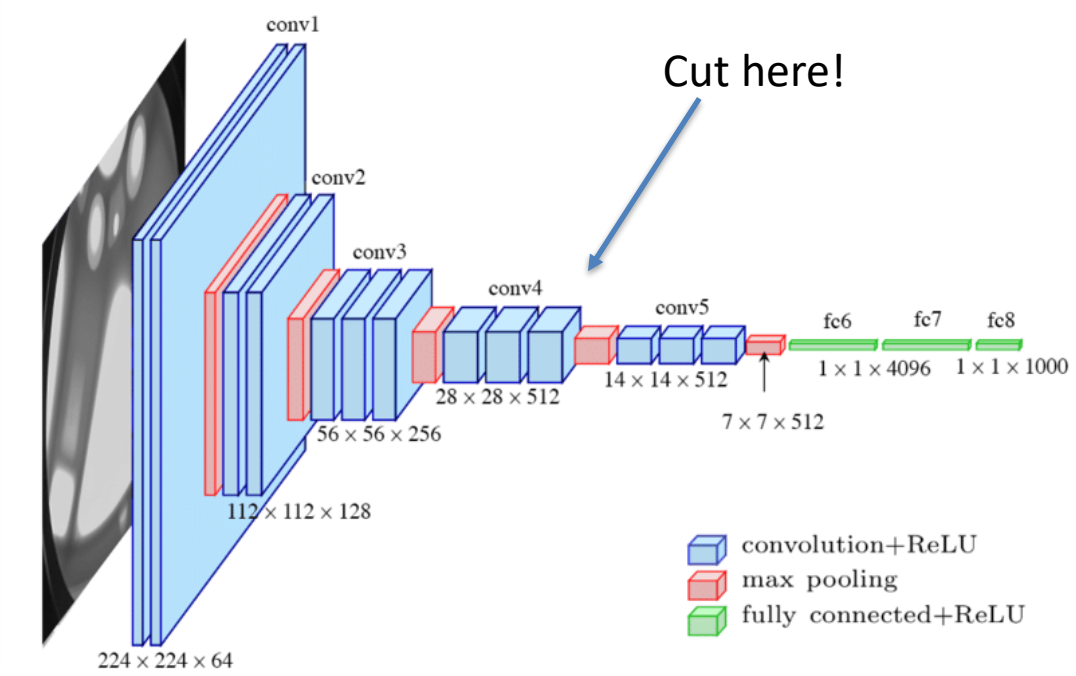
3) Models and Dilation Strategy



ConvTranspose2d: Source

VGG16

- Why 16 ? **13 Convolutional Layers and 3 Fully Connected**
- Decoder is not showing here: 3 ConvTranspose2d (512,256,128,64) and 2 Conv2d (64,64,19)

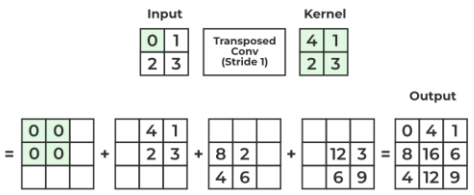


Just for visualization, sizes are not accurate!

Decoder is not showing here:
3 ConvTranspose2d
(512,256,128,64)

2 Conv2d (64,64,19)

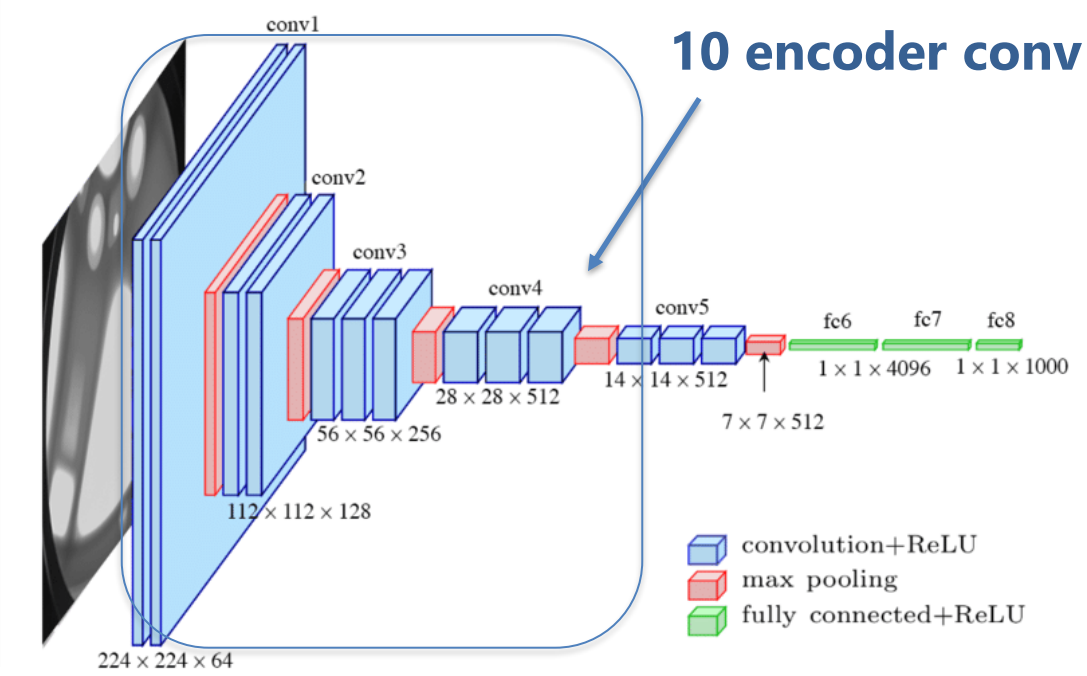
3) Models and Dilation Strategy



ConvTranspose2d: Source

Dilated VGG16

- Why 16 ? **13 Convolutional Layers and 3 Fully Connected**
- Decoder is not showing here: 3 ConvTranspose2d (512,256,128,64) and 2 Conv2d (64,64,19)



10 encoder conv layers : dilation =2

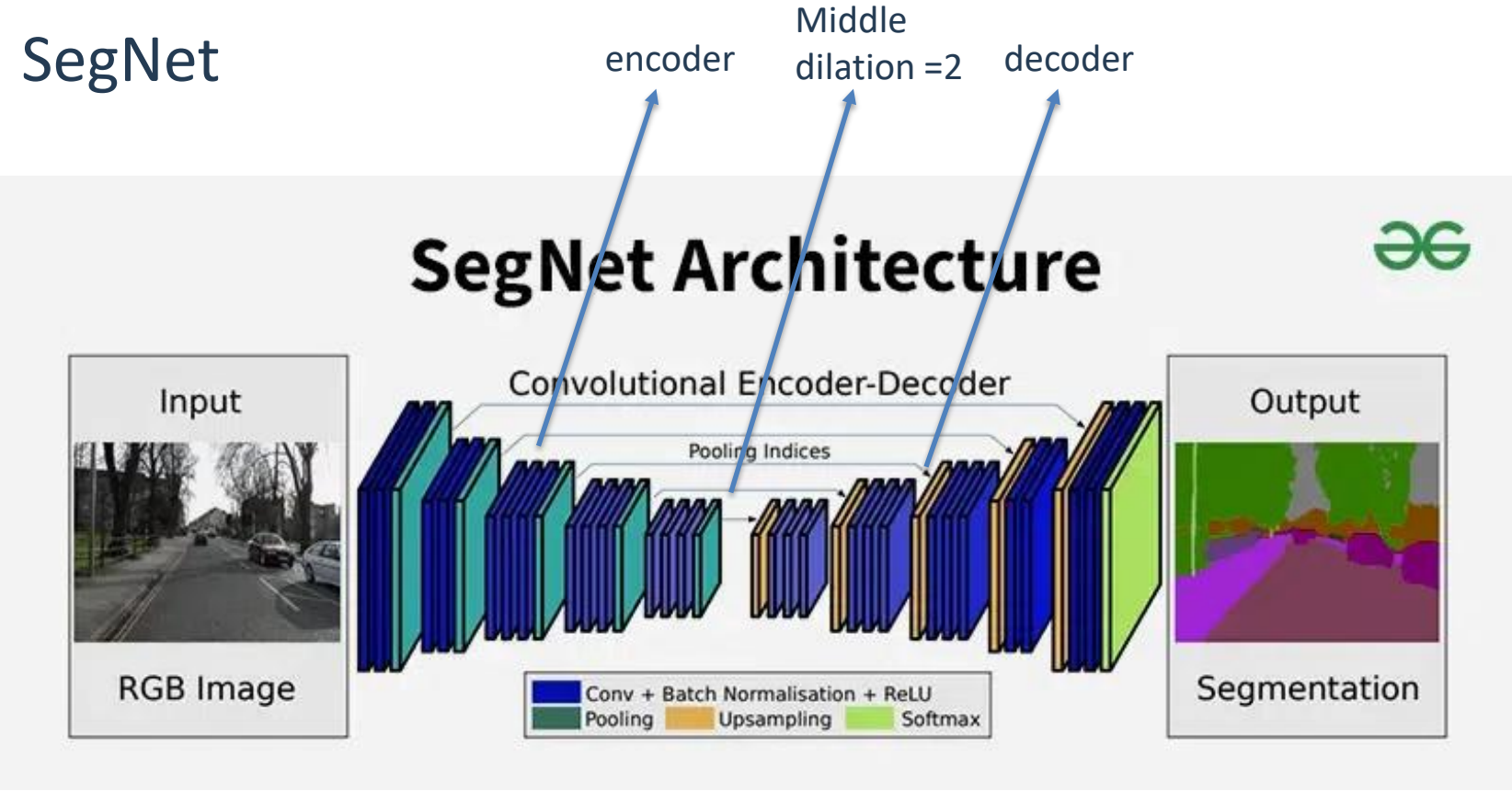
Decoder is not showing here:
3 ConvTranspose2d
(512,256,128,64)

2 Conv2d (64,64,19)

Just for visualization, sizes are not accurate!

3) Models and Dilation Strategy

SegNet



Architecture is not the same (Just for visualization)

Source: Link

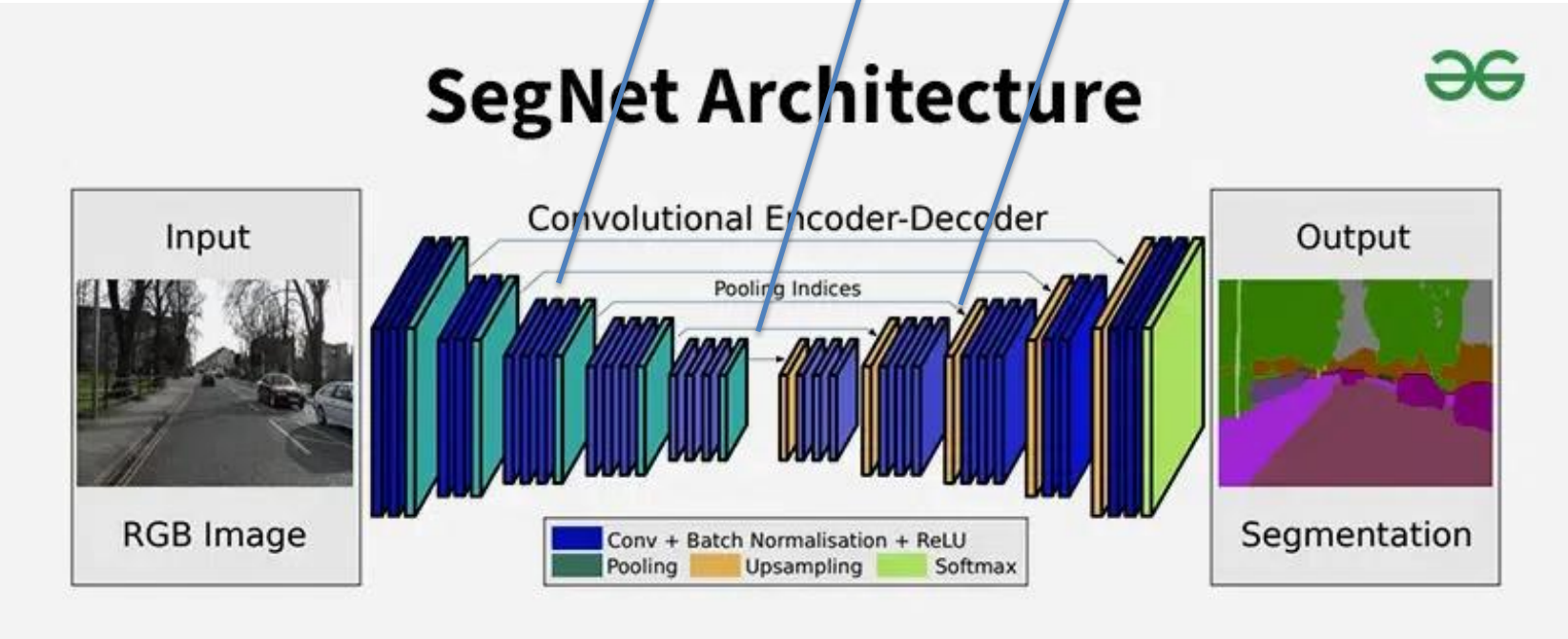
3) Models and Dilation Strategy

Dilated SegNet

encoder dilation =4

Middle dilation =~~2~~ 4

decoder dilation=4



Architecture is not the same (Just for visualization)

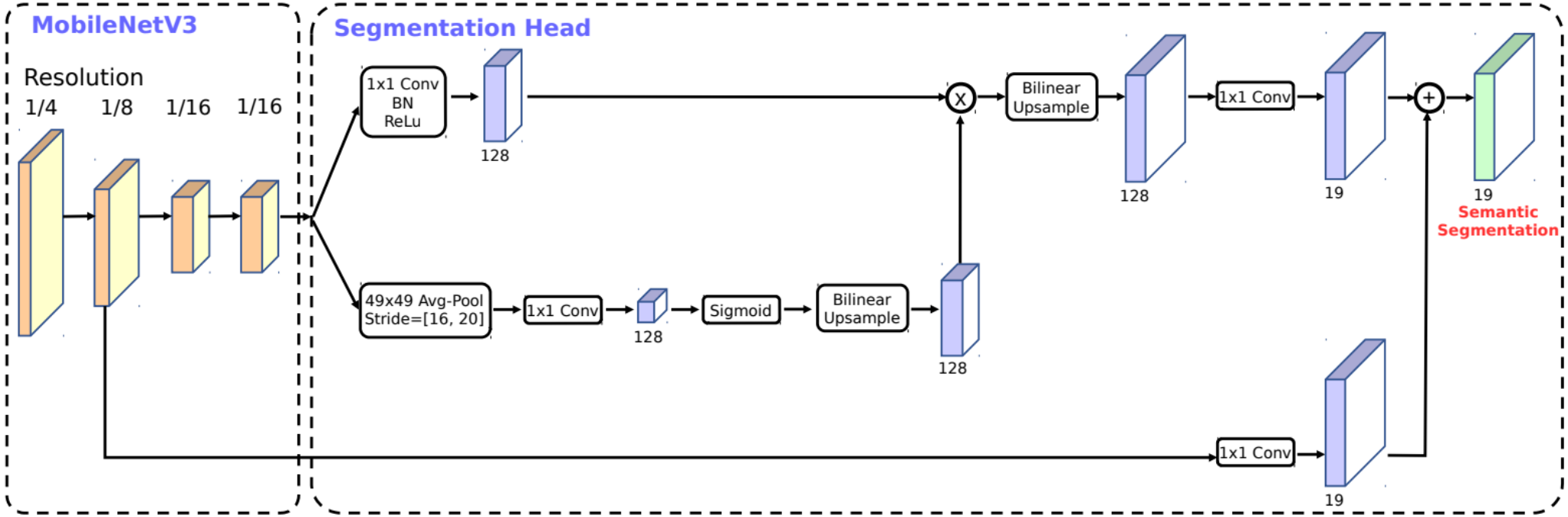
Source: Link

3) Models and Dilation Strategy

dilated convolution

LRASPP (Lite Reduced Atrous Spatial Pyramid Pooling)

- Lightweight decoder for mobile segmentation



Source: arxiv.org/pdf/1905.02244

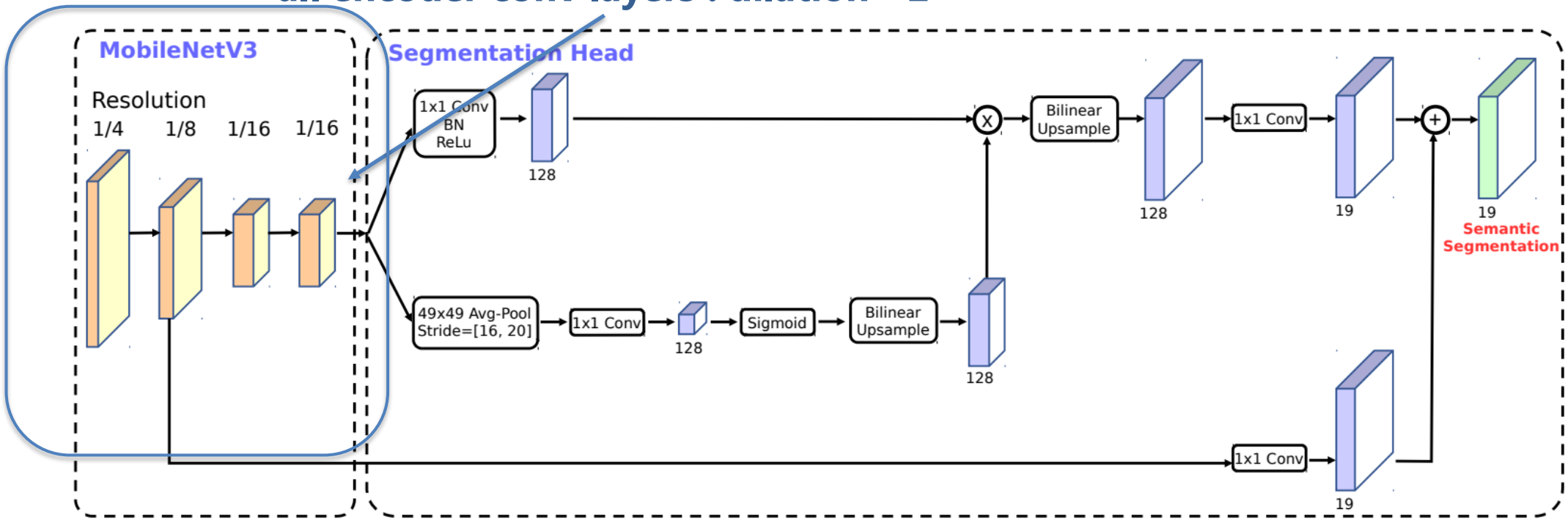
3) Models and Dilation Strategy

dilated convolution

LRASPP (Lite Reduced Atrous Spatial Pyramid Pooling)

- Lightweight decoder for mobile segmentation

all encoder conv layers : dilation =2



Source: arxiv.org/pdf/1905.02244

3) Training and Evaluation Setup

Component	Setting
Pairing each image with annotation	Torch built-in Cityscapes mapping
Loss	CrossEntropyLoss(ignore_index=255)
Optimizer	Adam (lr = 1e-4), 20 Epochs
Early stopping	Save best model by validation loss
Metric	mIoU
Evaluation	Validation split, first 100 samples.
Training/ Validation	2975/500 (Cityscapes predefined splits)
Batch size	2 to 5 ☹ (For $3\times1024\times2048$ image)

3) Mean IoU (Mean Intersection over Union)

- **mIoU** measures how much the predicted segmentation overlaps with the ground truth.
- It calculates IoU for each class and averages them across all classes.
- It is commonly used because it evaluates the quality of the predicted segmentation mask, not just pixel-wise accuracy (dominated by one or two class like road).

$$F1_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}$$

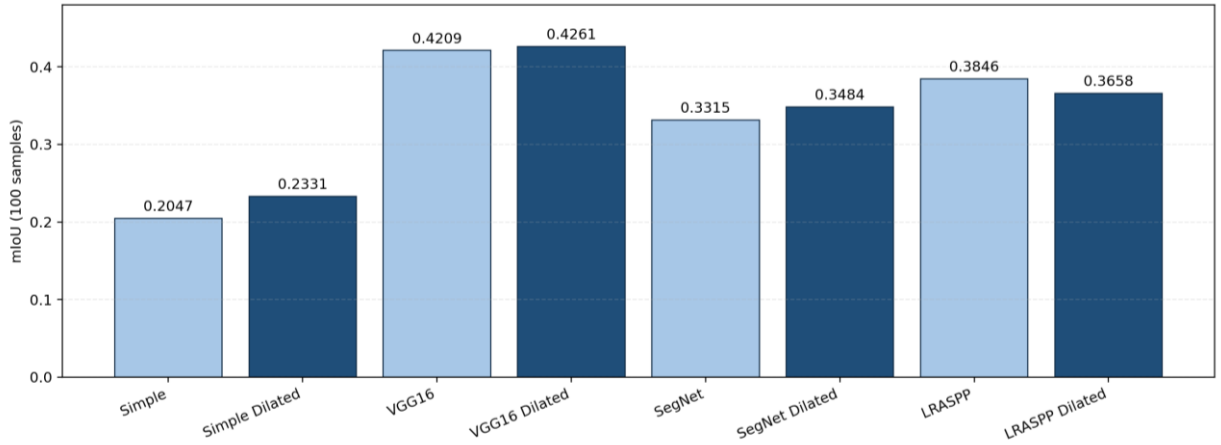
$$IoU_c = \frac{TP_c}{TP_c + FP_c + FN_c}$$

$$mIoU = \frac{IoU_1 + IoU_2 + IoU_3 + \cdots + IoU_{19}}{19}$$

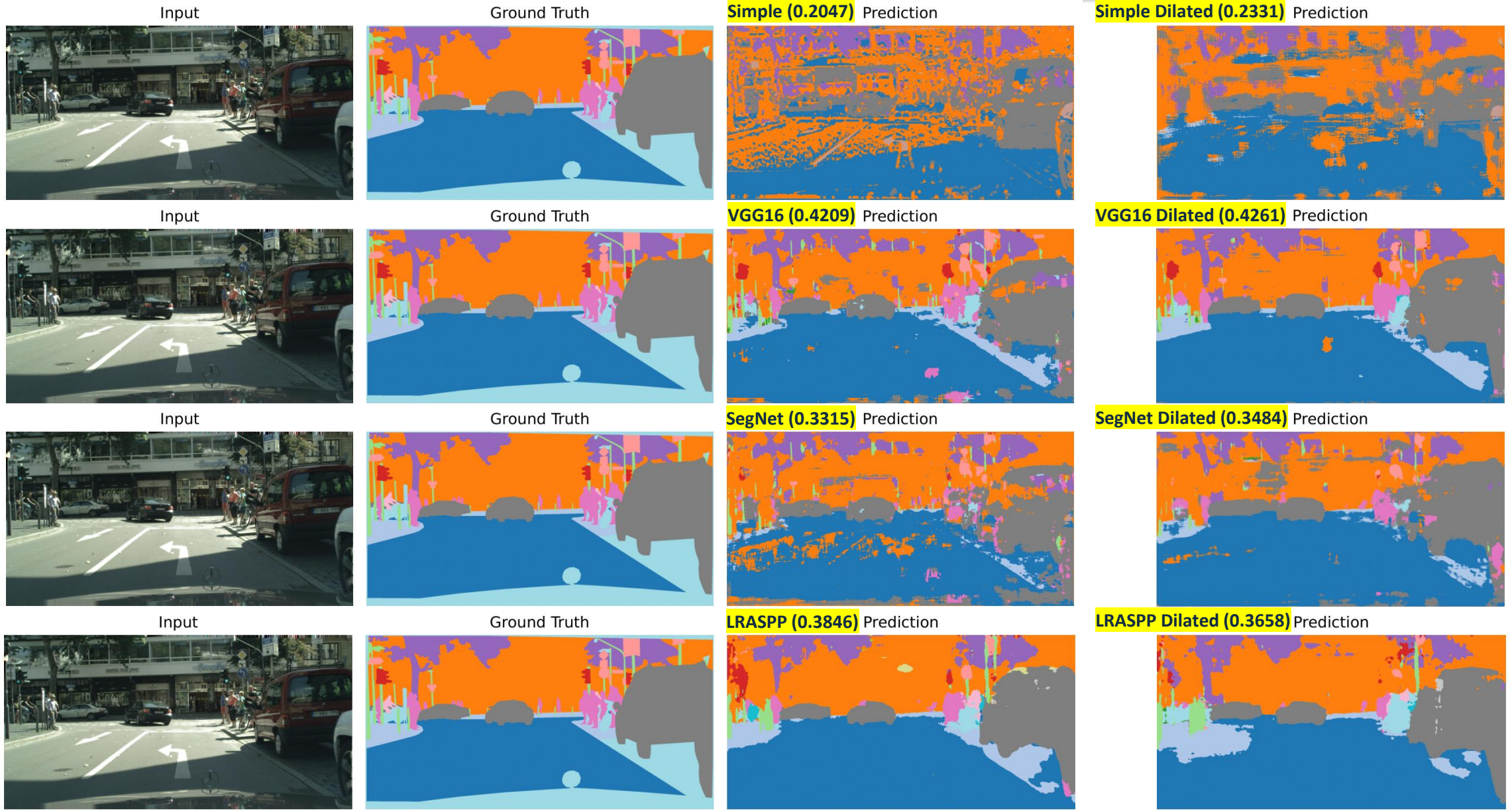
4) Mean IoU Results

Base	mIoU	Dilated	mIoU	Delta
Simple	0.2047	Simple Dilated	0.2331	+0.0284
VGG16	0.4209	VGG16 Dilated	**0.4261**	+0.0052
SegNet	0.3315	SegNet Dilated	0.3484	+0.0169
LRASPP	0.3846	LRASPP Dilated	0.3658	- 0.0188

Best overall: VGG16 Dilated (0.4261)
Largest gain: SegNet (+0.0169)
Largest drop: LRASPP (-0.0188)



4) Samples


Input
Ground Truth
Simple (0.2047) Prediction
Simple Dilated (0.2331) Prediction
Input
Ground Truth
VGG16 (0.4209) Prediction
VGG16 Dilated (0.4261) Prediction
Input
Ground Truth
SegNet (0.3315) Prediction
SegNet Dilated (0.3484) Prediction
Input
Ground Truth
LRASPP (0.3846) Prediction
LRASPP Dilated (0.3658) Prediction

4) Inference Results

Base	inference time for 10 samples	Number of parameters	Dilated	inference time for 10 samples	Number of parameters
Simple	0.1969	135,379	Simple Dilated	0.1970	135,379
VGG16	1.0597	8,367,379	VGG16 Dilated	1.0652	8,367,379
SegNet	0.5819	468,403	SegNet Dilated	0.6221	468,403
LRASPP	**0.1658**	3,221,198	LRASPP Dilated	0.1643	3,221,198

5) Conclusions and Lessons Learned

- Dilated convolutions improved 3 of 4 architecture families in this project.
- Best model: VGG16 Dilated (mIoU 0.4119). mIoU might not be a good measure (refer to the samples)
- Biggest positive effect of dilation: SegNet family (+0.0570 mIoU)
- Dilation is not always helpful: LRASPP dropped by 0.0211 mIoU
- Key lesson: tune dilation, see the max pooling and reducing in resolution in encoder before applying it

Base	mIoU	Dilated	mIoU	Delta
Simple	0.2047	Simple Dilated	0.2331	+0.0284
VGG16	0.4209	VGG16 Dilated	**0.4261**	+0.0052
SegNet	0.3315	SegNet Dilated	0.3484	+0.0169
LRASPP	0.3846	LRASPP Dilated	0.3658	- 0.0188

Base	inference time for 10 samples	Number of parameters	Dilated	inference time for 10 samples	Number of parameters
Simple	0.1969	135,379	Simple Dilated	0.1970	135,379
VGG16	1.0597	8,367,379	VGG16 Dilated	1.0652	8,367,379
SegNet	0.5819	468,403	SegNet Dilated	0.6221	468,403
LRASPP	**0.1658**	3,221,198	LRASPP Dilated	0.1643	3,221,198